

TC2037 Implementación de Métodos Computacionales

Actividad 3.2 Resaltador de sintaxis

(Evidencia de Competencias)

Esta actividad tiene como objetivo el que pongan en práctica los conocimientos adquiridos en la clase de al guiarlos en la implementación de un escáner y un parser de un pequeño lenguaje.

ESPECIFICACIÓN DEL LENGUAJE:

Las expresiones que conforman los programas en lenguajes funcionales como LISP y SCHEME que se analizarán léxica y sintácticamente para esta actividad pueden ser especificadas mediante la siguiente gramática libre de contexto en formato BNF (NO MODIFICAR LA GRAMÁTICA PARA ESCRIBIR EL PARSER):

```
<prog> ::= <exp> <prog> | $  
<exp> ::= <átomo> | <lista>  
<átomo> ::= símbolo | <constante>  
<constante> ::= número | booleano | string  
<lista> ::= ( <elementos> )  
<elementos> ::= <exp> <elementos> | ε
```

Donde los símbolos no terminales están escritos entre ángulos (<>) y los tipos de token (símbolos terminales) en **negritas** (ε representa la palabra vacía). O sea, un programa es una secuencia de 0 o más expresiones terminadas en \$, mientras que hay 2 tipos de expresiones: átomos y listas. Los átomos son de dos tipos: símbolos y constantes. Las listas se componen por 0 o más expresiones entre paréntesis.

Los **símbolos** son palabras compuestas por 1 o más letras minúsculas. Los **números** son secuencias de 1 o más dígitos. Los **booleanos** son #t o #f. Finalmente, las **strings** son secuencias de 0 o más letras minúsculas, dígitos o espacios entre comillas (""). Utilizar como delimitadores solamente a los espacios, los saltos de línea y los paréntesis.

Los caracteres blancos (espacios y saltos de línea) no son tokens, pero deben mantenerse para que la salida se vea indentada de forma similar que la entrada.

Los símbolos terminales deberán ser identificados y regresados por el analizador léxico (scanner) del lenguaje, mientras que el cumplimiento de la gramática deberá ser verificado por el analizador sintáctico (parser).

A partir del léxico y la sintaxis establecidos se pueden reconocer los programas bien contruidos, como el ejemplo de abajo. Pero también se deben de detectar errores en los programas erróneos como el ejemplo incorrecto de abajo.

Ejemplos correctos:

```
atomo 25 #t "hola amigo"
```

```
((1 2)(#f #t)) es una "lista anidada"
```

```
(define (prueba x)  
  (if (equal x 10)  
      (display (x igual a 10))  
      (display "x diferente de 10")))) $
```

Salida esperada:

```
atomo 25 #t "hola amigo"

((1 2)(#f #t)) es una "lista anidada"

(define (prueba x)
  (if (equal x 10)
      (display (x igual a 10))
      (display "x diferente de 10")))) $
```

Ejemplos incorrectos (al menos 1 error por línea ¿los reconocen?):

```
atomo 25 # $    ==> error de léxico en #

"hola @migo" $  ==> error de léxico en @migo

((1 2)#f #t)) es una lista anidada" $    ==> error de sintáxis en ))

(define (prueba x)
  (if (equal x 10)
      (display (x igual a10))
      (display "x diferente de 10")))) $ ==> error de sintáxis al final por falta de )
```

PARTE 1

Utiliza una herramienta de dibujo adecuada para dibujar el grafo dirigido del autómata finito determinista que diseñarás para reconocer los elementos de léxico del lenguaje solicitado.

PARTE 2

Tomando como referencia los programas de **Python** proporcionados en clase que implementan un analizador de léxico siguiendo la técnica de la matriz de transiciones (`obten_token.py`) y un analizador de sintaxis usando el método de descenso recursivo (`parser.py`) crea los tuyos para construir el escáner y el parser solicitados. Estos **no deben incluir** nada del código original que no sea necesario para esta actividad. El **código debe estar debidamente documentado internamente** mediante comentarios de Python e incluir tus datos de alumno como autor de los programas.

La interfaz de ejecución del programa es la siguiente:

- El programa deberá leer las expresiones desde el teclado.
- El programa realizará el análisis léxico y sintáctico sobre la entrada, y al mismo tiempo irá generando el código HTML que resaltará los elementos léxicos identificados mediante los colores más parecidos a los mostrados en el ejemplo de arriba, basado en las etiquetas definidas previamente por ustedes en un archivo de estilo CSS (`resalta_sintaxis.css`).
- Cuando se termine exitosamente de procesar la entrada completa, deberán completar el código HTML del código resaltado para que sea un archivo html que se pueda mostrar correctamente.
- Si el analizador detecta un error de léxico, deberá agregar la leyenda **">> ERROR LEXICO <<"**, completar el código HTML, para que la salida sea un archivo html correctamente formado y abortar la ejecución del programa.
- Si detecta un error de sintaxis, deberá hacer lo mismo que en el caso del error léxico, pero en este caso la leyenda será **">> ERROR SINTÁCTICO <<"**.

Documentación para entregar:

1. El código fuente de tu escáner en el archivo `obten_token.py`
2. El código fuente de tu parser en el archivo `parser.py`
3. Tu archivo de hoja de estilo en el archivo `resalta_sintaxis.css`

4. El reporte que se te solicita en la actividad en Canvas agregando el **AFD** que diseñaste para el léxico del lenguaje: **resalta_sintaxis.pdf**